



The Ultimate AI

LANGCHAIN CHEATSHEET

Your pocket guide to Agentic AI.



Basic Setup and Chat application

- Used to directly talk to an LLM (like ChatGPT) with a prompt.
- ChatOpenAI is the wrapper that connects LangChain to OpenAI chat models.
- temperature controls randomness (low = factual, high = creative).
- .invoke() sends a single prompt and returns a response object.
- Best for quick experiments, demos, or one-off prompts.

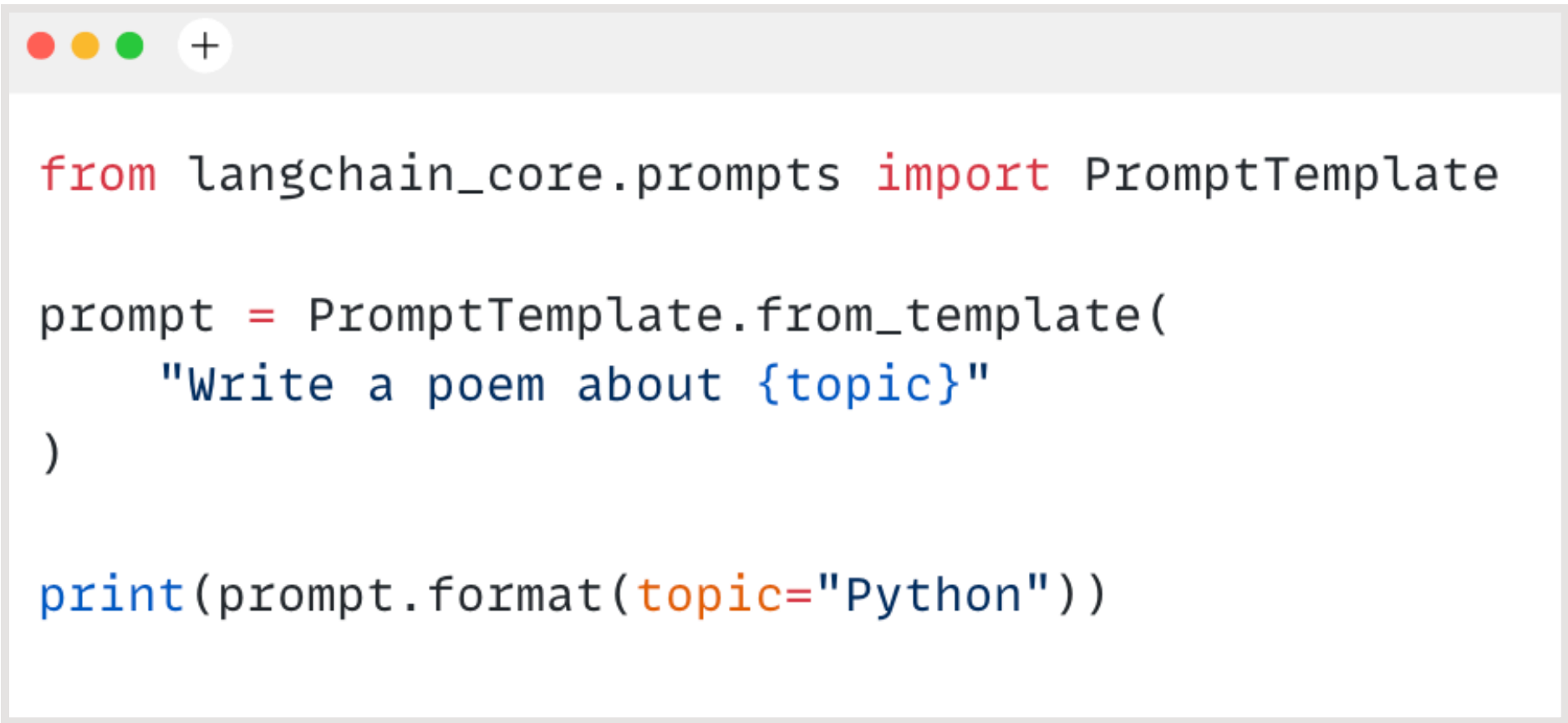
```
from langchain_openai import ChatOpenAI

chat = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.7
)

response = chat.invoke("What is LangChain?")
print(response.content)
```

PROMPT TEMPLATE

- Used to reuse prompts with dynamic values.
- Helps avoid hardcoding text into your code.
- Supports placeholders like {topic}, {concept}, etc.
- Makes prompts clean, readable, and maintainable.
- Ideal when the prompt structure stays same but inputs change.



```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Write a poem about {topic}"
)

print(prompt.format(topic="Python"))
```

PROMPT TEMPLATE

CHATPROMPTTEMPLATE

- Allows system + human + AI role-based prompts.
- Mimics how ChatGPT conversations work internally.
- Useful for instruction-following and persona control.
- Keeps prompts structured instead of long strings.
- Commonly used in production-grade chains.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert developer."),
    ("human", "Explain {topic} with a simple example.")
])

formatted = prompt.format_messages(topic="APIs")
print(formatted)
```

PROMPT TEMPLATE

STRUCTURED OUTPUT PROMPTS

- Forces the model to return output in a specific format (JSON, schema).
- Very useful when output needs to be parsed by code.
- Reduces hallucinated or inconsistent responses.
- Often combined with Output Parsers.
- Essential for agents, APIs, and automation workflows

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    """
    Provide output in JSON only.
    Name: {name}
    Age: {age}
    Return fields: summary, category
    """
)

print(prompt.format(name="Alice", age=28))
```

Chains

BASIC LLMCHAIN

- A Chain connects Prompt + LLM + Output.
- LLMChain is the classic LangChain abstraction.
- Supports single or multiple input variables.
- .run() is a shortcut for simple input usage.
- Good for linear, predictable workflows.

```
from langchain.chains import LLMChain
from langchain_core.prompts import PromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

prompt = PromptTemplate.from_template("Tell me a joke about {topic}")
chain = LLMChain(llm=llm, prompt=prompt)

print(chain.run("Python"))
```

LangChain LCEL

BASIC LCEL CHAIN

- LCEL = LangChain Expression Language.
- Uses | to connect components like Unix pipes.
- Much cleaner and more flexible than LLMChain.
- Each step does one job (prompt → model → parser).
- Recommended approach for modern LangChain apps.

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

chain = (
    PromptTemplate.from_template("Tell me a joke about {topic}.")
    | llm
    | StrOutputParser()
)

print(chain.invoke({"topic": "Python"}))
```

Agents and Tools

- Agents decide which tool to use and when.
- Tools are normal Python functions exposed to the LLM.
- Agent reasons step-by-step (ReAct pattern).
- Useful when the task requires decision-making, not just text.
- Common use cases: calculators, search, APIs, file operations.

```
from langchain.tools import Tool
from langchain.agents import initialize_agent, AgentType
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

# Simple tool
def calc(x: str) → str:
    # WARNING: eval() is unsafe with untrusted input.
    return str(eval(x))

calculator = Tool(
    name="Calculator",
    func=calc,
    description="Do basic math."
)

agent = initialize_agent(
    tools=[calculator],
    llm=llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION
)

print(agent.invoke({"input": "What is 5 * 12?"}))
```

Memory

- Stores conversation history.
- Allows the model to remember previous messages.
- Makes chats feel stateful and human-like.
- Useful for chatbots, assistants, tutors.
- Memory type controls how much context is retained.

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

conversation = ConversationChain(
    llm=llm,
    verbose=True,
    memory=ConversationBufferMemory()
)

print(conversation.predict(input="How can one overcome anxiety?"))
```

Document Loading

- Converts files into LangChain Document objects.
- Supports TXT, PDF, HTML, CSV, etc.
- First step for RAG (Retrieval-Augmented Generation).
- Usually followed by chunking and embeddings.
- Separates data ingestion from reasoning.

TEXT FILES

```
from langchain_community.document_loaders import TextLoader

docs = TextLoader("notes.txt").load()
```

PDF FILES

```
from langchain_community.document_loaders import PyPDFLoader

docs = PyPDFLoader("book.pdf").load()
```

Embeddings + Vector Store

- Embeddings convert text into numerical vectors.
- Vector stores enable semantic search, not keyword search.
- FAISS is a fast in-memory vector database.
- Used to find relevant chunks of documents.
- Core foundation of RAG systems.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# chunks should be a list of Documents (e.g., after splitting docs)
vector_store = FAISS.from_documents(chunks, embeddings)

results = vector_store.similarity_search("Explain LangChain", k=3)
print(results)
```

Retrieval-based Question Answering

- Combines retriever + LLM automatically.
- Retrieves relevant documents first.
- Then uses the LLM to generate an answer.
- Prevents hallucinations by grounding answers in data.
- Best for chatting with PDFs, docs, knowledge bases.

```
from langchain.chains import RetrievalQA

retriever = vector_store.as_retriever()

qa = RetrievalQA.from_chain_type(
    llm=model,                # model = ChatOpenAI(...)
    retriever=retriever,
    chain_type="stuff"
)

print(qa.invoke({"query": "What does the PDF say about AI?"})["result"])
```

Want more content like this?



Tap that follow button and stay in the loop!



Like



Comment



Share



Save